

PMIS INFOSYS LIMITED

Bhubaneswar Development Centre

Autonomous Cognitive Machine
(Cognix)

Software Handover & Architecture Document

Attribute	Details
Project Name	Autonomous Cognitive Machine (Cognix)
Delivery Unit / Location	Infosys Bhubaneswar Development Centre
Lab Allocation	Lab 3
Program	PMIS Internship Program
Team Members	1
Document Version	1.0 – Final
Date of Submission	June 2025
Classification	Confidential – Internal Use Only

Document Control

Revision History

Version	Date	Description of Change	Author	Approved By
1.0	June 2025	Initial handover document – Milestone 4 Final Delivery		PMIS Mentor

Distribution List

This document is intended for circulation to the following stakeholders. Recipients must treat the contents as confidential and refrain from external distribution without prior written authorization from Infosys Limited.

- Project Sponsor / Engagement Manager: PMIS Internship Program Office, Infosys Bhubaneswar
- Technical Lead / Architect: AI/ML Lab Supervisor, Lab 3
- Quality Assurance Lead: PMIS QA Reviewer
- PMIS Internship Program Coordinator: Infosys Bhubaneswar DC

Document Control.....	2
Revision History	2
Distribution List.....	2
1. Executive Summary	4
1.1 Project Snapshot.....	4
1.2 Objectives	4
1.3 Scope of This Document	4
2. System Architecture.....	5
2.1 High-Level Architecture Overview	5
2.2 LangGraph State Schema	5
2.3 Stateful Graph Architecture Flow.....	6
3. Core Modules Overview.....	7
3.1 Inter-Module Dependencies	7
4. Security & Context Isolation	8
4.1 Input Validation via Pydantic Schemas.....	8
4.2 Context Sandboxing – Sub-Agent Isolation	8
4.3 Secrets Management	9
5. Performance Optimizations.....	10
5.1 Context Offloading Strategy (Virtual File System).....	10
5.2 Execution Optimization – Anti-Loop Prompt Engineering.....	10
5.3 Additional Performance Measures.....	11
6. Deployment Guide	12
6.1 Prerequisites	12
6.2 Environment Variables	12
6.3 Containerization (Docker).....	12
6.4 Deployment Steps	13
6.5 Rollback Procedure	13
7. Testing Summary.....	14
7.1 End-to-End (E2E) Testing	14
7.2 Key Quality Metrics	14
7.3 Testing Tools & Environment.....	15
8. Challenges & Learnings.....	16
8.1 Technical Challenges Encountered	16
8.2 Key Learnings	16
8.3 Recommendations for Future Enhancements.....	17
Annexure A: Glossary of Terms	18
Document Sign-Off	19

1. Executive Summary

This document constitutes the formal Software Handover and Architecture record for the Autonomous Cognitive Machine (Cognix), prepared as part of the PMIS Internship Program executed at the Infosys Bhubaneswar Development Center, Lab 4 & Lab 5. It is intended to provide senior stakeholders, incoming engineering teams, and quality assurance personnel with a complete and authoritative reference covering the system’s stateful graph architecture, core functional modules, AI security posture, performance engineering decisions, deployment procedure, and validation outcomes.

ACM has been engineered as a stateful, multi-agent cognitive system capable of autonomously executing multi-day research and deep reasoning tasks. Using a LangGraph-based cyclic StateGraph, the platform overcomes the fundamental limitations of standard LLM deployments—specifically context degradation, hallucination, and memory loss across extended task horizons. The purpose of this handover is to ensure continuity of ownership, minimize onboarding friction for future contributors, and establish a single source of truth for all architectural and operational decisions made during the four-milestone development lifecycle.

1.1 Project Snapshot

Attribute	Details
Project Name	Autonomous Cognitive Machine (Cognix)
Domain	Artificial Intelligence & Multi-Agent Systems
Technology Stack	Python 3.11+, LangGraph, LangChain, Mistral (LLM), LangSmith,
Team Size	1 (AI/ML Intern)
Project Duration	8 Weeks (40 Working Days)
Delivery Status	Production Ready – Milestone 4 Completed
Hosting Environment	Local / Cloud Containerized Deployment

1.2 Objectives

- **Primary Objective:** To build a stateful, multi-agent cognitive architecture capable of executing multi-day research and reasoning tasks without human intervention, context degradation, or hallucination.
- **Target User Base:** Research professionals, data analysts, and enterprise teams requiring deep, autonomous investigation across large information corpora.
- **Key Differentiators:** Native stateful memory via LangGraph StateGraph, autonomous task decomposition via TODO planning, and a Virtual File System (VFS) for context offloading—capabilities not present in standard single-turn LLM deployments.

1.3 Scope of This Document

The scope of this handover document encompasses the stateful graph architecture, the four core functional modules, the AI security and context isolation framework, performance optimization strategies, the local and containerized deployment process, and a summary of LangSmith-validated testing outcomes. It excludes day-to-day operational runbooks and detailed API contracts, which are maintained separately in the project’s source code repository and LangSmith project workspace.

2. System Architecture

Cognix is designed using a state-driven multi-agent architecture implemented through the LangGraph framework. The system uses a shared state object to store user input, generated TODO tasks, current task progress, intermediate results, virtual memory files, and final outputs. The Planner Agent analyzes the user goal and generates structured tasks, while the Executor Agent processes each task and sends it through the Delegation Layer to the appropriate specialized agent such as the Research Agent, Code Agent, or Summarizer Agent. After execution, the Reviewer Agent validates the generated outputs, and the Synthesizer Agent combines all validated results into a final structured report that can be exported in PDF and TXT formats.

2.1 High-Level Architecture Overview

Architecture Style & Rationale

Cognix is designed around a Stateful Multi-Agent Graph Architecture implemented using the LangGraph framework, where each workflow node represents a specific agent action such as planning, execution, reviewing, or synthesis. Unlike traditional single-response AI systems, Cognix maintains a shared state throughout the workflow to store user goals, generated TODO tasks, intermediate outputs, virtual memory files, and final reports. This allows the system to break a user goal into smaller tasks, process them through specialized agents, validate the outputs, and generate a structured final report in an organized and scalable manner.

The main design rationale of Cognix is based on three key requirements:

- State Persistence:** The system uses a shared state to store user input, generated tasks, task status, intermediate results, and virtual memory files across the workflow.
- Multi-Agent Task Execution:** Cognix divides responsibilities among specialized agents such as Planner, Executor, Research, Code, Summarizer, Reviewer, and Synthesizer to improve modularity and efficiency.
- Structured Workflow Control:** LangGraph manages the workflow by moving tasks through planning, execution, delegation, validation, and synthesis in a controlled and organized manner.

2.2 LangGraph State Schema

In Cognix, the LangGraph State Schema acts as the central memory structure of the system. It stores all important execution data such as user input, generated TODO tasks, current task status, intermediate results, virtual memory files, retry information, and final reports. Each agent reads from and updates this shared state during workflow execution, allowing smooth communication between Planner, Executor, Reviewer, Synthesizer, and other specialized agents.

State Field	Python Type	Purpose & Description
messages	List[BaseMessage]	Stores the user goal or input message.
todos	List[str]	Stores the generated TODO tasks with their descriptions and status.
file_system_dict	Dict[str, str]	Acts as a virtual memory system where task outputs and final reports are stored.
results	List[str]	Stores intermediate outputs generated by agents.

2.3 Stateful Graph Architecture Flow

The diagram below describes the layered architecture of **Cognix**. Each layer maps to a distinct functional responsibility within the LangGraph execution workflow.

Stateful Graph Architecture – ACE Layer Map

[User Objective (Natural Language Input)]

↓

■ **ORCHESTRATION LAYER (LangGraph StateGraph)** — Defines graph topology, node routing logic, conditional edges, and workflow control.

The entry point triggers the Planner Node, followed by Executor, Reviewer, and Synthesizer nodes.

↓ ↑ (cyclic edges)

■ **REASONING LAYER (Grok LLM via LangChain)** — The Planner, Summarizer, Research, Code, Reviewer, and Synthesizer agents run here.

Mistral AI processes the current task, generates responses, supports reasoning, creates code, summarizes information, and prepares final structured outputs. ↓ ↑

■ **STATE / MEMORY LAYER (Python TypedDict)** — The shared LangGraph state schema stores messages, todos, current_task, files, results,

retry_count, and other execution data. All nodes read from and write updates to this layer.

This layer serves as the persistent working memory of the Cognix workflow. ↓

■ **OBSERVABILITY LAYER (LangSmith)** — LLM calls, agent executions, tool invocations, and workflow transitions can be traced

using LangSmith for monitoring, debugging, and performance analysis.

Layer	Responsibility	Technology / Framework
Orchestration Layer	Graph topology, node routing, conditional workflow edges, task loop control, and multi-agent workflow execution	LangGraph StateGraph
Reasoning Layer	Natural language understanding, task planning, code generation, summarization, research response generation, validation, and final response creation	Mistral AI, LangChain
State / Memory Layer	Persistent in-memory storage of user messages, generated TODO tasks, task status, intermediate results, virtual memory files, and final reports	Python State Dictionary / TypedDict, LangGraph State
Observability Layer	Execution tracing, agent monitoring, prompt and response tracking, debugging support, and performance analysis	LangSmith

3. Core Modules Overview

Cognix is composed of multiple core modules, each responsible for a specific function in the multi-agent workflow. These modules collaborate through LangGraph and shared state to plan tasks, execute them, validate outputs, and generate final reports.

Module	Description	Key Features	Technology / Notes
Task Planning Engine	Converts the user goal into structured TODO tasks that guide the overall execution workflow.	Goal understanding, task decomposition, TODO generation, task status initialization	LangChain, Mistral AI, Python
Virtual File System (VFS)	Stores intermediate task outputs and final reports inside the shared state as virtual files.	Task output storage, task_1.txt creation, final report storage, memory persistence during execution	Python Dictionary, LangGraph State
Sub-Agent Delegation	Routes each task to the appropriate specialized agent based on task requirements.	Task classification, routing to Research Agent, Code Agent, or Summarizer Agent, dynamic execution flow	LangGraph Runnables, Python, LangChain Tool Binding, Mistral AI
Integration & Synthesis	Combines validated outputs from all tasks and generates the final structured report.	Output aggregation, final report synthesis, PDF/TXT generation, clean result presentation	Mistral AI, ReportLab, Python

3.1 Inter-Module Dependencies

Module Dependency Map – ACE Execution Flow
Task Planning Engine (Module 1) → produces todos[] ↓ Consumed by Executor Agent through the LangGraph workflow
Virtual File System (Module 2) ↔ Executor Agent writes/reads intermediate task outputs ↓ Stored data is used by agents and final synthesis process
Sub-Agent Delegation (Module 3) ← Executor Agent sends tasks through the Delegation Layer ↓ Sub-agent outputs are returned and stored in the shared state
Integration & Synthesis (Module 4) ← reads all virtual memory files from state["files"]

4. Security & Context Isolation

Security in Cognix is designed mainly for AI workflow stability, data integrity, safe agent execution, and API key protection rather than traditional web authentication. Since Cognix is a Streamlit-based multi-agent AI system, the main security concerns include prompt injection, incorrect agent routing, malformed model outputs, cross-agent state contamination, runtime errors, and credential exposure. The system reduces these risks through controlled task routing, shared state management, environment-based secret handling, and isolated agent execution.

4.1 Input Validation via Pydantic Schemas

Input Validation & Anti-Injection Framework

- In Cognix, user input is first processed by the Planner Agent, which converts the goal into structured TODO tasks. These tasks are then executed through the Executor Agent and routed by the Delegation Layer. Instead of allowing agents to directly modify the full system state, Cognix controls execution through predefined agent functions and structured task flow.
- The system validates and controls execution by ensuring that tasks follow a defined workflow: planning, execution, delegation, validation, and synthesis. This helps prevent malformed outputs or unexpected execution behavior from disrupting the LangGraph workflow.

Control	Mechanism	Threat Mitigated
Task Flow Validation	Tasks are generated as structured TODO items with descriptions and status values	Unstructured task execution, incomplete workflow
Controlled Agent Routing	Delegation logic routes tasks only to predefined agents such as Research, Code, or Summarizer Agent	Incorrect agent execution, uncontrolled tool usage
Output Cleaning	Generated outputs are cleaned before storage to remove unwanted HTML or malformed text	UI formatting issues, unwanted HTML artifacts

4.2 Context Sandboxing – Sub-Agent Isolation

Each specialized agent in Cognix performs a specific role within a controlled execution context. The Research Agent handles research-related outputs, the Code Agent handles programming tasks, and the Summarizer Agent generates concise explanations. These agents do not directly control the entire workflow; instead, they receive only the task description from the Delegation Layer and return their output as text.

The Executor Agent is responsible for storing outputs in the shared state and virtual file system. This prevents sub-agents from directly modifying global workflow data and reduces the risk of cross-agent state contamination.

Context Isolation Controls

- **Limited Input Passing:**
Each sub-agent receives only the task description required for execution, not the full system state.

- **Dedicated Agent Responsibility:**
Each agent has a specific role such as research, code generation, summarization, reviewing, or synthesis.
- **Output-Only Integration:**
Sub-agent outputs are returned as plain text and are stored by the Executor Agent in `state["files"]`.
- **Shared State Protection:**
The shared state is updated only through controlled workflow nodes such as Planner, Executor, Reviewer, and Synthesizer.
- **Delegation Control:**
The Delegation Layer ensures that tasks are routed only to approved agents.

4.3 Secrets Management

- **Secure API Key Management:** API keys such as Mistral, Tavily, and LangSmith keys are stored in environment variables using a `.env` file instead of being hardcoded in the source code.
- **Shared State Protection:** The system uses a controlled shared state to store tasks, results, virtual files, and final outputs, ensuring that only defined workflow nodes update the execution data.
- **Agent Context Isolation:** Specialized agents such as Research Agent, Code Agent, and Summarizer Agent receive only the required task description, preventing unnecessary access to the full system context.
- **Controlled Task Delegation:** The Delegation Layer routes tasks only to predefined agents, reducing the risk of incorrect or unintended agent execution.
- **Output Cleaning and Validation:** Generated outputs are cleaned and reviewed before final synthesis to reduce formatting issues, unwanted HTML artifacts, and inconsistent results.

5. Performance Optimizations

Performance in Cognix is measured by two primary dimensions: API token efficiency and execution reliability. Since Cognix uses multiple agents and LLM calls during planning, execution, delegation, review, and synthesis, performance optimization is important to reduce response time, avoid unnecessary API usage, and prevent repeated execution loops. The optimizations below address both cost efficiency and workflow stability.

5.1 Context Offloading Strategy (Virtual File System)

Primary Performance Optimization: VFS Context Offloading

Standard LLM-based systems may become inefficient during long workflows because repeated reasoning steps can increase context size. When multiple task outputs are included directly in the active prompt history, the system may experience higher token usage, slower execution, rate-limit errors, or incomplete responses.

Cognix reduces this issue by using a **Virtual File System** through state["files"]. Instead of keeping every generated output inside the active reasoning context, task results are stored as virtual files such as task_1.txt, task_2.txt, and final_report.txt.

Cognix follows a **write-store-retrieve** pattern:

- 4. Write:**
After each task is completed, the Executor Agent stores the generated output into state["files"].
- 5. Store:**
The intermediate output is saved as a virtual file and does not need to be repeatedly included in every active LLM call.
- 6. Retrieve:**
During final synthesis, the Synthesizer Agent reads all stored task outputs and combines them into a final structured report.

- Token Efficiency:**
By storing intermediate results in the virtual file system, Cognix reduces unnecessary context size during repeated execution steps.
- State Persistence:**
Intermediate task outputs are preserved inside the shared LangGraph state, ensuring data is not lost between workflow nodes.
- Retrieval Efficiency:**
Reading from the virtual file system is faster than repeatedly regenerating content using additional LLM API calls.

5.2 Execution Optimization – Anti-Loop Prompt Engineering

Optimization Technique	Implementation	Performance Impact
Restrictive System Prompts	The Planner, Summarizer, Code Agent, and Synthesizer prompts define clear output rules such as task count, response format, and completion requirements.	Reduces unnecessary long responses, repeated outputs, and wasted API calls.

Strict Delegation Boundaries	The Delegation Layer routes each task only to predefined agents such as Research Agent, Code Agent, or Summarizer Agent.	Prevents incorrect agent execution and avoids unnecessary tool or model calls.
TODO-Driven Termination	The workflow checks the todos list and moves to the Synthesizer only when all tasks are marked as done.	Provides a deterministic exit condition and prevents premature termination or repeated execution.
LangSmith Trace Monitoring	LangSmith can be used to monitor agent execution, model calls, workflow transitions, and errors during testing.	Helps identify slow steps, repeated calls, routing issues, and performance bottlenecks.

5.3 Additional Performance Measures

- Model Optimization:**
 Cognix can use lightweight models such as mistral-small for frequent tasks like planning and summarization, while larger models can be reserved for final synthesis when higher-quality output is required.
- Task Count Control:**
 The system limits the number of generated TODO tasks to avoid excessive API calls and reduce execution time.
- Output Length Control:**
 Prompts are designed to generate concise and complete responses, preventing unnecessarily long outputs and reducing token usage.
- Rate Limit Handling:**
 Delays, retry logic, and smaller models can be used to reduce rate-limit errors during multiple agent calls.
- Efficient State Storage:**
 Intermediate task outputs are stored in state["files"], reducing repeated processing and improving workflow efficiency.
- Streamlit UI Optimization:**
 Heavy visual effects such as excessive blur, large shadows, and repeated rendering can be reduced to improve UI responsiveness.

6. Deployment Guide

This section provides the operational instructions required to reproduce, configure, and deploy the Cognix platform. All commands have been validated in the development environment at Infosys Bhubaneswar DC.

6.1 Prerequisites

- **Runtime Environment:** Python 3.11 or higher is recommended for compatibility with LangGraph, LangChain, Streamlit, and modern Python type annotations.
- **Package Manager:** `pip` is used for installing project dependencies. A virtual environment is recommended to isolate dependencies.
- **Mistral API Key:** Active Mistral API key is required for LLM-based task planning, summarization, code generation, and synthesis.
- **Tavily API Key:** Active Tavily Search API key is required for research and external information retrieval.
- **LangSmith API Key:** Active LangSmith API key is required if workflow tracing, monitoring, and debugging are enabled.
- **Version Control:** Git CLI can be used for repository management and version control.
- **Container Platform:** Optional: Docker Engine 24.x+ can be used for containerized deployment.

6.2 Environment Variables

The following environment variables must be defined in a `.env` file at the project root prior to any execution. Sensitive values must never be committed to version control.

Variable Name	Description	Example / Expected Value
MISTRAL_API_KEY	Authentication credential for the Mistral AI API. Loaded at runtime using <code>python-dotenv</code> .	xxxxxxxxxxxxxxxxxxxx
TAVILY_API_KEY	Authentication credential for Tavily Search API used by the Research Agent.	tvly-xxxxxxxxxxxxxxxx
LANGSMITH_API_KEY	Authentication credential for LangSmith tracing and observability.	Lsv2_xxxxxxxxxxxxxx
LANGCHAIN_TRACING_V2	Enables LangSmith trace streaming. Set to <code>true</code> if LangSmith monitoring is required.	true
LANGCHAIN_PROJECT	Identifies the LangSmith project workspace to which traces are submitted.	Cognix

6.3 Containerization (Docker)

Docker can be used for containerized deployment of the Cognix system. This allows the application to run consistently across different environments.

Docker Build & Run Commands

```
# Build the Docker image
```

```
docker build -t Cognix-ai:latest .  
  
# Run with environment variables from .env file  
docker run --env-file .env -p 8501 cognix-ai:late
```

6.4 Deployment Steps

7. Clone or download the Cognix project folder to the target machine.
8. Open the project folder in Visual Studio Code or any preferred code editor.
9. Create a .env file in the project root directory and add the required API keys such as Mistral API Key, Tavily API Key, and LangSmith API Key if monitoring is enabled.
Install all required dependencies globally using pip: `pip install -r requirements.txt`
10. Verify that all required libraries such as Streamlit, LangGraph, LangChain, Mistral AI, Tavily, ReportLab, and python-dotenv are installed successfully.
11. Run the Streamlit application using the command: `streamlit run app.py`
12. Open the local Streamlit URL displayed in the terminal to access the Cognix user interface.
13. Enter a user goal or research topic in the input field and start the workflow execution.
14. If LangSmith tracing is enabled, open the LangSmith dashboard to monitor agent execution, workflow traces, prompts, responses, and performance details.
15. After execution, view the generated TODO tasks, virtual memory files, final report, and download the report in PDF or TXT format.

6.5 Rollback Procedure

Rollback Strategy

16. **Code Rollback:**
If a recent code change causes errors, the project can be restored to the last working version using Git or a backup copy of the project folder. This helps recover stable versions of files such as app.py, graph.py, agent modules, and tool files.
17. **Dependency Rollback:**
Since the project uses a global Python environment, dependency issues can be resolved by reinstalling the previously working package versions. If required, the requirements.txt file can be restored from the last stable version and dependencies can be reinstalled using:
`pip install -r requirements.txt`
18. **Environment Configuration Rollback:**
If API key or environment variable issues occur, the .env file can be checked and restored with the correct Mistral, Tavily, and LangSmith API keys. This ensures that external services continue working properly.
19. **Application Restart**
Cognix uses an in-memory shared state during execution. If a workflow fails, the current session can be restarted by stopping the Streamlit server and running the application again: `streamlit run app.py`
20. **LangSmith Trace Review**
If LangSmith tracing is enabled, failed executions can be reviewed in the LangSmith dashboard to identify errors, performance issues, or incorrect agent behavior before restarting the workflow.

7. Testing Summary

Comprehensive testing was performed to validate the correctness, reliability, workflow execution, and output quality of the Autonomous Cognitive Machine (Cognix) system. Testing focused on verifying whether the system could accept user goals, generate structured tasks, execute tasks through specialized agents, validate outputs, and generate final reports in PDF and TXT formats.

The testing process covered functional testing, agent-level testing, workflow testing, report generation testing, and observability testing using LangSmith. The objective was to ensure that all agents and modules work together correctly within the LangGraph-based multi-agent workflow.

7.1 End-to-End (E2E) Testing

Parameter	Details
Testing Tool / Framework	Manual testing through Streamlit UI, LangGraph workflow execution, and LangSmith tracing
Test Coverage (% of Modules)	100% – Planner, Executor, Research Agent, Code Agent, Summarizer Agent, Reviewer Agent, Synthesizer Agent, Virtual File System, and Report Generation modules tested
Total Test Scenarios Executed	Multiple test scenarios using research topics, coding tasks, summarization tasks, and final report generation
Pass / Fail Summary	Majority of workflows executed successfully with task generation, agent execution, validation, and report generation completed
Key Scenarios Validated	User goal submission, TODO task generation, task execution, sub-agent delegation, virtual memory file creation, reviewer validation, final synthesis, PDF/TXT download
Resolution Applied	Fixed task indexing in Executor, cleaned HTML artifacts using output filtering, optimized model usage to reduce rate-limit issues, and improved delegation logic for Code Agent routing
Critical Defects Identified	Task-result mismatch, HTML tag artifacts in generated output, API rate-limit errors, and incorrect agent routing after adding Code Agent

7.2 Key Quality Metrics

Metric	Target / Threshold	Observed Result
Task Decomposition Accuracy	More than 80%	Planner Agent generated meaningful and structured TODO tasks for most user goals

Agent Routing Accuracy	More than 80%	Delegation Layer correctly routed tasks to Research, Code, or Summarizer agents after improving routing conditions
Task Completion Rate	More than 80%	Executor successfully processed pending tasks and updated task status to done
Report Generation Success	100% for completed runs	Final reports were generated successfully in TXT and PDF formats
Output Validation Quality	More than 80%	Reviewer Agent verified task completion and helped maintain output consistency
UI Usability	User-friendly and responsive	Streamlit interface allowed goal submission, task display, memory file viewing, and report downloading
Monitoring Support	Traceable workflow execution	LangSmith supported tracing, debugging, and workflow monitoring when enabled

7.3 Testing Tools & Environment

- **Evaluation Framework:** LangSmith Tracing Platform – primary evaluation and trace inspection tool.
- **Quality Assessment:** LLM-as-a-Judge via Mistral – qualitative scoring of agent reasoning quality and report completeness.
- **Test Environment:** Local development environment at Infosys Bhubaneswar DC, Lab 4 & 5.
- **Test Data Strategy:** Synthetic long-form research corpora generated specifically for context-limit stress testing; multi-source document sets with overlapping and contradictory content for summarization accuracy assessment.

8. Challenges & Learnings

*This section describes the major technical challenges encountered during the development of the **Autonomous Cognitive Machine (Cognix)**, along with the solutions applied and the key learnings gained from the project. The challenges were mainly related to multi-agent workflow control, task-result mapping, API rate limits, output formatting, agent delegation, and report generation.*

8.1 Technical Challenges Encountered

- **Challenge 1 – API Rate Limit Issues**

During execution, the system made multiple API calls through different agents such as the Planner Agent, Summarizer Agent, Code Agent, and Synthesizer Agent. When large models were used frequently, the system sometimes faced API rate-limit errors.

Resolution:

Model usage was optimized by using lightweight models for frequent tasks and reserving larger models for final synthesis when required. Task count control and better prompt design were also applied to reduce unnecessary API calls.

- **Challenge 2 – Task and Result Mismatch:**

In earlier implementation, task outputs were sometimes saved under incorrect task file names. This caused mismatch between generated TODO tasks and their corresponding results in the virtual memory files.

Resolution:

The Executor Agent logic was improved by using proper task indexing while saving outputs. Each task result was stored with its correct file name such as task_1.txt, task_2.txt, and so on. This ensured correct task-result mapping.

- **Challenge 3 – Incorrect Agent Routing:**

- After adding the Code Agent, some research-based tasks were incorrectly routed to the Code Agent because the delegation logic used broad keyword matching.

Resolution:

The Delegation Layer was refined with stricter routing conditions. Code-related tasks were routed to the Code Agent only when clear code-related phrases such as “write code,” “generate code,” or “python code” appeared in the task description.

8.2 Key Learnings

- **Learning 1 – Clear Agent Routing is Essential**

In a multi-agent system, delegation logic must be carefully designed. If routing conditions are too broad, tasks may be assigned to the wrong agent. Strict and clear routing rules improve execution accuracy.

- **Learning 2 – Shared State is the Backbone of Multi-Agent Workflows**

The shared state plays a key role in storing user input, generated tasks, task progress, outputs, virtual files, and final reports. Proper state management ensures smooth communication between agents.

- **Learning 3 – Context Sandboxing Improves AI Reasoning Quality**

Well-defined prompts help agents generate more accurate, concise, and structured outputs. Prompt rules such as output format, word limits, and task boundaries improve system reliability.

8.3 Recommendations for Future Enhancements

- **Recommendation 1 – AI-Based Agent Routing**

Instead of relying only on keyword-based routing, future versions can use an AI-based classifier to decide whether a task should go to the Research Agent, Code Agent, or Summarizer Agent.

- **Recommendation 2 – Persistent Memory Storage**

Currently, Cognix uses in-memory shared state and virtual files. Future versions can integrate a database such as SQLite, Redis, or MongoDB to store task history and reports permanently.

- **Recommendation 3 – Parallel Agent Execution**

The current workflow processes tasks sequentially. Future improvements can include parallel execution of independent tasks to reduce total execution time.

Annexure A: Glossary of Terms

Term	Definition
Cognix	Autonomous Cognitive Machine – the multi-agent AI system developed in this project for task planning, execution, validation, and report generation.
LangGraph	An open-source Python framework for building stateful, cyclic multi-agent applications using a directed graph model.
LangChain	A Python framework providing abstractions for LLM interactions, tool binding, and prompt management, used here to interface with the Grok API.
Mistral AI	The large language model (LLM) used as the reasoning engine for both the Supervisor and sub-agents in the Cognix platform.
LangSmith	A tracing and observability platform for LangChain/LangGraph applications, used for execution monitoring and LLM-as-a-judge evaluation.
LLM	Large Language Model – an AI model capable of understanding natural language, generating text, summarizing information, and producing code.
StateGraph	The core LangGraph class that defines a cyclic directed graph where nodes are agent actions and edges are state transitions.
Shared State	The central memory object used in Cognix to store user input, TODO tasks, current task status, generated outputs, virtual files, retry count, and final reports.
Virtual File System	A memory-based file storage mechanism implemented using state["files"]. It stores intermediate task outputs such as task_1.txt, task_2.txt, and final_report.txt.
Tavily API	An external search API used by the Research Agent to retrieve relevant and updated information.
Streamlit	A Python framework used to build the interactive user interface for Cognix. It allows users to enter goals, view tasks, inspect outputs, and download reports.
Planner Agent	Creates TODO tasks from the user goal.
Executor	Executes tasks step by step.
Delegation Layer	Routes tasks to the correct specialized agent.
Synthesizer	Combines all outputs into the final report.
PMIS	Project Management Internship / IT Skills – the Infosys internship program under which this project was developed.

Document Sign-Off

By signing below, the undersigned acknowledge that the contents of this handover document have been reviewed and accepted as an accurate representation of the Autonomous Cognitive Engine (ACE) system as of the date of submission.

Role	Name	Designation	Signature / Date
Intern / Author		AI/ML Intern, PMIS	
PMIS Mentor / Reviewer		Technical Lead, Infosys BDC	
Program Coordinator		PMIS Program Office	